SET COVER

Formalización Problema Auditoria

Jacobo Arroyave Ortiz, Camilo Niño Galvez

Facultad de Inteligencia Artificial e Ingenierías Universidad de Caldas

Análisis y diseño de algoritmos

Doc. Luz Enith Guerrero Mendieta

Noviembre 05, 2026

SET COVER

Subproyecto 3 (Parte 2): Cobertura Mínima de Auditoría Normativa

Alcance

Este documento formaliza la **Parte 2 del Subproyecto 3**: seleccionar el número mínimo de etapas o interacciones de un flujo de trabajo para auditarlas de forma que todas las categorías de riesgo normativo queden cubiertas al menos una vez.

El documento sigue el estándar formal del curso: definición rigurosa del problema de optimización y su versión de decisión (**AUDIT-COVER-DEC**), tamaño de la entrada, demostración de pertenencia a la clase NP, prueba de NP-dureza mediante la reducción polinomial estándar desde **Cobertura de Vértices** (**VERTEX-COVER-DEC**, Proposición 4.3.1), deducción analítica de la función de eficiencia $T(n, r)$ para el algoritmo exacto (fuerza bruta) y para el algoritmo aproximado voraz, así como la fundamentación formal de su cota de aproximación armónica H_r.

---

1. Formalización del problema

1.1 Parámetros y tamaño de la entrada

El crecimiento del problema está gobernado por dos dimensiones principales:

- $n \in \mathbb{Z}_{\geq 1}$: número de interacciones o etapas candidatas a ser auditadas en el flujo de trabajo.
- $r \in \mathbb{Z}_{\geq 1}$: número de categorías de riesgo normativo distintas que deben cubrirse (por ejemplo: sesgo, privacidad, formato, seguridad, trazabilidad, etc.).

1.2 Entradas

Una instancia del problema está definida por la tupla $(\mathcal{U}, \mathcal{I})$:

1. **Universo de categorías de riesgo ($\mathcal{U}$):** un conjunto finito de r categorías de riesgo identificables:

$$\mathcal{U} = \{R_1, R_2, \ldots, R_r\}, \quad \text{con } |\mathcal{U}| = r.$$

2. **Colección de interacciones candidatas ($\mathcal{I}$):** una familia indexada de n subconjuntos de $\mathcal{U}$:

$$\mathcal{I} = \{S_1, S_2, \dots, S_n\}, \quad \text{donde } S_i \subseteq \mathcal{U} \text{ para todo } i \in \{1, \dots, n\}.$$

Cada conjunto S_i representa el subconjunto de categorías de riesgo que quedan cubiertas si se audita la interacción (etapa) i-ésima.

(En el problema de decisión se añade además un entero $k \in \mathbb{Z}_{\geq 0}$, con $0 \leq k \leq n$, que representa el presupuesto máximo permitido de auditorías).

1.3 Salidas

- **Versión de optimización (**AUDIT-COVER-OPT**):** un subconjunto de índices de interacciones seleccionadas $C \subseteq \{1, \dots, n\}$ (o equivalentemente la subcolección $\{S_i\}_{i \in C}$) que cubra la totalidad de $\mathcal{U}$ con cardinalidad mínima $|C|$, o la indicación None (infactible) si $\bigcup_{i=1}^{n} S_i \neq \mathcal{U}$.
- **Versión de decisión (**AUDIT-COVER-DEC**):** un valor booleano (**SÍ** / **NO**) que responde si existe algún subconjunto $C \subseteq \{1, \dots, n\}$ que cumpla las restricciones con a lo sumo k interacciones.

1.4 Restricciones

Toda solución candidata $C \subseteq \{1, \dots, n\}$ debe satisfacer simultáneamente:

1. **Restricción de dominio binario (selección 0/1):** cada interacción puede auditarse a lo sumo una vez. Si definimos variables de decisión $x_i \in \{0,1\}$ tales que $x_i = 1 \Leftrightarrow i \in C$, entonces:

$$x_i \in \{0,1\} \quad \forall i \in \{1, \dots, n\}.$$

2. **Restricción de cobertura total:** cada categoría de riesgo $R_j \in \mathcal{U}$ debe ser cubierta por al menos una de las interacciones seleccionadas en C:

$$\bigcup_{i \in C} S_i = \mathcal{U} \quad \Leftrightarrow \quad \sum_{i : R_j \in S_i} x_i \geq 1 \quad \forall R_j \in \mathcal{U}.$$

3. **Condición necesaria de factibilidad global:** el problema admite solución si y solo si la unión de todas las interacciones disponibles cubre el universo:

$$\bigcup_{i=1}^{n} S_i = \mathcal{U}.$$

1.5 Objetivo

- **Versión de optimización (AUDIT-COVER-OPT):** minimizar la cantidad total de interacciones auditadas:

$$\min_{C \subseteq \{1,\dots,n\}} |C| = \min_{x \in \{0,1\}^n} \sum_{i=1}^{n} x_i \quad \text{sujeto a } \bigcup_{i \in C} S_i = \mathcal{U}.$$

- **Versión de decisión (AUDIT-COVER-DEC):** dado un umbral entero $k \in \mathbb{Z}_{\geq 0}$: > ¿Existe un subconjunto de interacciones $C \subseteq \{1, \dots, n\}$ tal que >

$$\bigcup_{i \in C} S_i = \mathcal{U} \qquad \text{y} \qquad |C| \leq k?$$

1.6 Representación y tamaño formal de la entrada

En memoria, la instancia se puede codificar como una matriz de incidencia binaria de dimensión $n \times r$ (donde la entrada (i, j) vale 1 si $R_j \in S_i$ y 0 en caso contrario) o como n listas de identificadores de riesgo donde cada $|S_i| \leq r$ y cada identificador requiere $O(\log r)$ bits.

En consecuencia, el tamaño de la entrada medido como número de elementos estructurales es:

$$N_{\text{elem}} = |\mathcal{U}| + n + L, \qquad L = \sum_{i=1}^{n} |S_i| \leq n \cdot r,$$

es decir, $N_{\text{elem}} = O(n \cdot r)$.

La longitud de la entrada en bits depende de la representación utilizada. Mediante una matriz de incidencia binaria se requieren $O(n \cdot r)$ bits para representar las relaciones de cobertura (sin contar pequeños costos de delimitación y codificación). Mediante listas de identificadores, la longitud es

$$N_{\text{bits}} = O(r + n + L\log(r+1) + \log(k+1)), \qquad L = \sum_{i=1}^{n} |S_i| \leq n \cdot r.$$

En ambos casos, el tamaño de la representación es polinomial en n y r. Además, como $k \leq n$, el valor numérico del umbral está acotado por el número de interacciones; a diferencia de la Parte 1, donde el presupuesto B puede ser exponencial en su número de bits, aquí no hay un parámetro numérico que pueda crecer independientemente del tamaño estructural de la instancia.

---

2. Pertenencia de AUDIT-COVER-DEC a la clase NP

Proposición 1. AUDIT-COVER-DEC ∈ NP.

2.1 Certificado

Sea $I = (\mathcal{U}, \mathcal{I}, k)$ una instancia de AUDIT-COVER-DEC. Un certificado de solución positiva consiste en un subconjunto de índices de interacciones:

$$C \subseteq \{1, \ldots, n\}.$$

Como $|C| \leq n$, el certificado tiene tamaño $O(n)$ (o $O(n \log n)$ bits), que es polinomial respecto al tamaño de la entrada $O(n \cdot r)$.

2.2 Algoritmo verificador en tiempo polinomial

El verificador determinista $V(I, C)$ ejecuta los siguientes pasos:

1. **Verificación de cardinalidad:** comprueba que $C \subseteq \{1, \ldots, n\}$ y que $|C| \leq k$. Esta operación toma tiempo $O(|C|) = O(n)$.
2. **Construcción de la cobertura:** inicializa un conjunto vacío $K \leftarrow \emptyset$ y, para cada índice $i \in C$, acumula los riesgos cubiertos haciendo $K \leftarrow K \cup S_i$. Como $|C| \leq n$ y cada conjunto S_i tiene a lo sumo r elementos, realizar a lo sumo n uniones sobre un universo de tamaño r requiere a lo sumo $O(n \cdot r)$ operaciones elementales.
3. **Verificación de cobertura completa:** comprueba si $\mathcal{U} \subseteq K$ (es decir, si $|K| = r$ dado que $K \subseteq \mathcal{U}$). Esto toma tiempo $O(r)$.
4. **Decisión:** retorna **SÍ** si ambas condiciones ($|C| \leq k$ y $K = \mathcal{U}$) se cumplen; de lo contrario retorna **NO**.

El tiempo total del verificador es:

$$T_{\text{verif}}(n, r) = O(n) + O(n \cdot r) + O(r) = O(n \cdot r),$$

que es polinomial en el tamaño de la entrada. Por lo tanto, AUDIT-COVER-DEC ∈ NP.

---

3. Demostración de NP-Dureza mediante reducción polinomial desde Cobertura de Vértices

Para demostrar que AUDIT-COVER-DEC es NP-difícil se presenta la reducción estándar de la **Proposición 4.3.1**:

$$\text{VERTEX-COVER-DEC} \leq_P \text{AUDIT-COVER-DEC}.$$

La idea central es interpretar un grafo como una instancia de auditoría: **cada arista es una categoría de riesgo** que debe quedar cubierta y **cada vértice es una interacción** que, al auditarse, cubre exactamente las aristas que inciden en él. Basta con una única reducción válida desde un problema NP-completo para establecer la NP-dureza.

3.1 Problema de origen: VERTEX-COVER-DEC

VERTEX-COVER-DEC es uno de los 21 problemas NP-completos de Karp (1972) (Garey & Johnson, 1979, problema GT1).

- **Entrada:**
  - Un grafo simple no dirigido $G = (V, E)$, con $V = \{v_1, \ldots, v_p\}$ y $E = \{e_1, \ldots, e_m\}$, donde cada arista es un par no ordenado $e = \{u, w\}$ con $u, w \in V$ y $u \neq w$.
  - Un entero $q \in \mathbb{Z}_{\geq 0}$.
- **Pregunta:** ¿Existe un subconjunto de vértices $D \subseteq V$ con $|D| \leq q$ tal que toda arista tenga al menos un extremo en D?

$$\forall\, e = \{u, w\} \in E: \quad u \in D \ \vee \ w \in D.$$

Un conjunto D con esta propiedad se denomina **cobertura de vértices** de G.

Supuesto sobre las instancias de origen. Se consideran únicamente grafos con al menos una arista ($|E| \geq 1$), de modo que la instancia transformada cumpla $r \geq 1$ (Sección 1.1). Esta restricción no afecta la NP-completitud de VERTEX-COVER-DEC: si $E = \emptyset$, la

respuesta es trivialmente **SÍ** para todo $q \geq 0$ (basta $D = \emptyset$) y puede decidirse en tiempo $O(|V|)$ antes de aplicar la reducción.

3.2 Construcción de la transformación polinomial f

Dada una instancia arbitraria $I_{\text{VC}} = (G, q)$ con $G = (V, E)$, $|V| = p$ y $|E| = m$, se construye $f(I_{\text{VC}}) = I_{\text{AUDIT}} = (\mathcal{U}, \mathcal{I}, k)$ así:

1. **Universo de categorías de riesgo (aristas → riesgos):** se crea una categoría de riesgo R_e por cada arista $e \in E$:
$$\mathcal{U} = \{R_e : e \in E\}, \qquad r = |\mathcal{U}| = m.$$
2. **Interacciones candidatas (vértices → interacciones):** se crea una interacción por cada vértice $v_i \in V$, de modo que
$$n = p.$$
3. **Categorías cubiertas por cada interacción (aristas incidentes → cobertura):** la interacción asociada a v_i cubre exactamente las categorías correspondientes a las aristas incidentes a v_i:
$$S_i = \{R_e : e \in E,\ v_i \in e\} \subseteq \mathcal{U}, \qquad |S_i| = \deg(v_i).$$
4. **Umbral de decisión:** para respetar el dominio $0 \leq k \leq n$ definido en la Sección 1.2, se fija
$$k = \min(q, p).$$
   - Si $q \leq p$, entonces $k = q$.
   - Si $q > p$, la instancia de Vertex Cover es trivialmente **SÍ** (basta $D = V$, con $|D| = p < q$), y la instancia de auditoría también es **SÍ**, pues seleccionar las $n = p = k$ interacciones cubre todo $\mathcal{U}$ (toda arista tiene sus extremos en V). La equivalencia se demuestra formalmente para ambos casos en la Sección 3.4.

La familia $\mathcal{I} = (S_1, \ldots, S_n)$ se trata como una **familia indexada**: aunque dos vértices tengan el mismo conjunto de aristas incidentes (por ejemplo, dos vértices aislados con $S_i = S_j = \emptyset$), corresponden a interacciones distintas $i \neq j$. Así, la correspondencia vértice ↔ interacción es una biyección entre V y $\{1, \ldots, n\}$.

Propiedad clave de la construcción. Para toda arista $e \in E$ y todo vértice $v_i \in V$:

$$R_e \in S_i \Leftrightarrow v_i \in e \Leftrightarrow v_i \text{ es un extremo de } e.$$

En consecuencia, cada categoría R_e con $e = \{u, w\}$ pertenece exactamente a dos conjuntos: el de la interacción asociada a u y el de la asociada a w.

3.3 Complejidad de la transformación

Con el grafo representado mediante listas de adyacencia (o lista de aristas):

- Crear las m categorías de riesgo R_e cuesta $O(m)$.
- Crear las p interacciones vacías cuesta $O(p)$.
- Recorrer cada arista $e = \{u, w\}$ una sola vez e insertar R_e en S_u y en S_w cuesta $O(1)$ por arista, es decir, $O(m)$ en total. El tamaño total de la instancia producida es

$$L = \sum_{i=1}^{n} |S_i| = \sum_{v \in V} \deg(v) = 2m$$

  por el lema del apretón de manos.
- Calcular $k = \min(q, p)$ requiere una comparación; en el modelo de bits, copiar el valor resultante cuesta tiempo lineal en la longitud de su representación binaria, es decir, $O(\log(q + 1))$.

Por lo tanto:

$$T_f(p, m) = O(p + m) = O(|V| + |E|),$$

lineal en el tamaño de la instancia original. Si se prefiere la representación por matriz de incidencia $n \times r$, la construcción cuesta $O(|V| \cdot |E|)$, que sigue siendo polinomial. En ambos casos, f es computable en **tiempo polinomial**.

3.4 Demostración de equivalencia bidireccional (⇔)

Se demuestra que $I_{\text{VC}} = (G, q)$ tiene respuesta **SÍ** si y solo si $I_{\text{AUDIT}} = f(I_{\text{VC}})$ tiene respuesta **SÍ**.

Dirección $(\Rightarrow)$***: si G tiene una cobertura de vértices de tamaño*** $\leq q$***, la instancia de auditoría es SÍ***

Supóngase que existe $D \subseteq V$ con $|D| \leq q$ que es cobertura de vértices de G. Se define la selección de interacciones

$$C = \{i \in \{1, \dots, n\} : v_i \in D\}.$$

1. **Límite de auditorías:** como la correspondencia vértice ↔ interacción es biyectiva, $|C| = |D|$. Además, $|D| \leq q$ por hipótesis y $|D| \leq |V| = p$ porque $D \subseteq V$. Por tanto,
$$|C| = |D| \leq \min(q, p) = k.$$
2. **Cobertura total:** sea $R_e \in \mathcal{U}$ arbitraria, con $e = \{u, w\}$. Como D es cobertura de vértices, $u \in D$ o $w \in D$; sin pérdida de generalidad, $u = v_{i^*} \in D$, de modo que $i^* \in C$. Por $(*)$, como v_{i^*} es extremo de e, se tiene $R_e \in S_{i^*}$. Luego
$$R_e \in \bigcup_{i \in C} S_i.$$
Al valer para toda $R_e \in \mathcal{U}$, y como cada $S_i \subseteq \mathcal{U}$, se concluye $\bigcup_{i \in C} S_i = \mathcal{U}$.

Por tanto, C certifica que AUDIT-COVER-DEC responde **SÍ**.

Dirección $(\Leftarrow)$***: si la instancia de auditoría es SÍ, G tiene una cobertura de vértices de tamaño*** $\leq q$

Supóngase que existe $C \subseteq \{1, \ldots, n\}$ con $|C| \leq k$ y $\bigcup_{i \in C} S_i = \mathcal{U}$. Se define el conjunto de vértices

$$D = \{v_i \in V : i \in C\}.$$

1. **Cardinalidad:** por la biyección vértice ↔ interacción,
$$|D| = |C| \leq k = \min(q, p) \leq q.$$
2. **Cobertura de aristas:** sea $e = \{u, w\} \in E$ arbitraria. Como $R_e \in \mathcal{U} = \bigcup_{i \in C} S_i$, existe $i^* \in C$ tal que $R_e \in S_{i^*}$. Por $(*)$, v_{i^*} es un extremo de e, es decir, $v_{i^*} \in \{u, w\}$. Como $i^* \in C$, se tiene $v_{i^*} \in D$, y por tanto $u \in D \vee w \in D$.

Así, D es una cobertura de vértices de tamaño a lo sumo q, y VERTEX-COVER-DEC responde **SÍ**.

3.5 Conclusión de NP-dureza sobre los componentes del proyecto

Se ha demostrado que f es computable en tiempo polinomial y que

$$(G, q) \in \text{VERTEX-COVER-DEC} \Leftrightarrow f(G, q) \in \text{AUDIT-COVER-DEC}.$$

Por consiguiente:

$$\text{VERTEX-COVER-DEC} \leq_P \text{AUDIT-COVER-DEC}.$$

1. **NP-dureza de la versión de decisión:** como VERTEX-COVER-DEC es NP-completo y se reduce polinomialmente a AUDIT-COVER-DEC, este último es NP-**difícil**.
2. **NP-completitud:** combinando lo anterior con AUDIT-COVER-DEC ∈ NP (Sección 2), se concluye que **AUDIT-COVER-DEC es** NP**-completo**.
3. **NP-dureza de la versión de optimización:** si existiera un algoritmo polinomial que calculara el tamaño mínimo $k^* = |C^*|$ de una cobertura de auditoría, bastaría comparar $k^* \leq k$ para resolver AUDIT-COVER-DEC en tiempo polinomial. Por tanto, **AUDIT-COVER-OPT es** NP**-difícil**, y no existe un algoritmo exacto de tiempo polinomial para él a menos que P = NP.
4. **Componente afectado:** la conclusión se refiere específicamente al **componente de selección mínima de interacciones a auditar** (Parte 2). No implica que cada operación o módulo del sistema sea NP-difícil: por ejemplo, verificar una cobertura dada cuesta $O(n \cdot r)$ (Sección 2) y el algoritmo voraz corre en tiempo polinomial.
5. **Implicación sobre la arquitectura:** el módulo exacto (src/auditoria/brute_force.py) tiene costo exponencial en el peor caso y solo es viable para instancias pequeñas de verificación; para instancias medianas y grandes se utiliza el módulo voraz (src/auditoria/greedy.py), de tiempo polinomial y con garantía de aproximación (Sección 5).

Observación. En la instancia construida, cada interacción cubre $|S_i| = \deg(v_i)$ categorías. Como VERTEX-COVER-DEC sigue siendo NP-completo en grafos con grado máximo 3 (Garey, Johnson & Stockmeyer, 1976), el problema de auditoría permanece NP-difícil **incluso cuando cada interacción cubre a lo sumo 3 categorías de riesgo**, lo cual conecta este resultado con la variante *3-Set Cover* mencionada en el enunciado.

---

4. Algoritmo exacto (Fuerza Bruta) y deducción formal de su eficiencia

En src/auditoria/brute_force.py, el algoritmo exacto explora los subconjuntos de interacciones en orden creciente de tamaño $k = 1,2,\ldots,n$. El primer subconjunto encontrado cuya unión cubra $\mathcal{U}$ es garantizadamente óptimo, pues todos los subconjuntos de tamaño menor a k ya fueron descartados.

4.1 Conteo de operaciones y deducción de $T_{\text{exacto}}(n, r)$

Analicemos el código de fuerza_bruta(universo, interacciones) paso a paso en el peor caso (cuando el algoritmo debe examinar todos los tamaños, por ejemplo, porque la única cobertura usa las n interacciones o porque la instancia es infactible):

1. **Bucle externo sobre el tamaño de combinación:** for k in range(1, n + 1) itera desde $k = 1$ hasta $k = n$.
2. **Generación de combinaciones de tamaño k :** la función recursiva combinaciones(nombres, k) construye explícitamente la lista de las $\binom{n}{k}$ combinaciones de tamaño k. Como la recursión no poda ramas sin elementos suficientes, cada nodo del árbol de recursión corresponde a un subconjunto de tamaño $j \leq k$, de modo que el árbol tiene $\sum_{j=0}^{k} \binom{n}{j} \leq 2^n$ nodos; además, cada una de las $\binom{n}{k}$ hojas copia la combinación (actual.copy()) en $O(k)$. El costo de generación para un tamaño k es, por tanto, $O\left(2^n + k\binom{n}{k}\right)$, y sumado sobre todos los tamaños:

   $$\sum_{k=1}^{n} O\left(2^n + k\binom{n}{k}\right) = O(n \cdot 2^n) + O(n \cdot 2^{n-1}) = O(n \cdot 2^n).$$

   La lista de cada tamaño se almacena completa en memoria, lo que implica un espacio de $O\left(k\binom{n}{k}\right)$ para el tamaño k que se está procesando.
3. **Verificación de cada combinación:** para cada una de las $\binom{n}{k}$ combinaciones:
   - Se inicializa cubiertas = set() en tiempo $O(1)$.
   - El bucle interno for nombre in combinacion: se ejecuta exactamente k veces.
   - En cada iteración del bucle interno, cubiertas.update(interacciones[nombre]) inserta $|S_i| \leq r$ categorías de riesgo, con costo $O(r)$ en el peor caso. Por tanto, construir la unión para una combinación de tamaño k cuesta $O(k \cdot r)$.
   - La comprobación universo.issubset(cubiertas) cuesta $O(r)$.
   - La impresión de la combinación y su unión (print(combinacion, "->", cubiertas)) cuesta $O(k + r)$, que no altera la cota.

Las operaciones sobre set se analizan bajo el supuesto habitual de costo promedio $O(1)$ por elemento para las operaciones hash.

SET COVER

En el peor caso, cada combinación de tamaño k requiere procesar hasta k conjuntos, cada uno con un máximo de r categorías. Por tanto, el costo de verificar una combinación es $O(k \cdot r)$. Sumando sobre todos los tamaños posibles y agregando el costo total de generación:

$$T_{\mathrm{FB}}(n,r) = \underbrace{O(n \cdot 2^n)}_{\text{generación}} + \sum_{k=1}^{n} \binom{n}{k} O(k \cdot r) = O(n \cdot 2^n) + O\left(r \sum_{k=1}^{n} k \binom{n}{k}\right).$$

Utilizando la identidad combinatoria clásica

$$\sum_{k=1}^{n} k \binom{n}{k} = n \cdot 2^{n-1},$$

se obtiene, como $r \geq 1$, la cota superior de peor caso:

$$\boxed{T_{\mathrm{FB}}(n,r) = O(n \cdot 2^n) + O(r \cdot n \cdot 2^{n-1}) = O(n \cdot r \cdot 2^n).}$$

Esta es una cota **superior** de peor caso: el costo real de cada combinación depende de cuántas categorías contienen sus conjuntos, por lo que $O(k \cdot r)$ no necesariamente se alcanza en todas las instancias. El crecimiento exponencial se manifiesta cuando el algoritmo debe examinar una cantidad exponencial de combinaciones antes de encontrar una solución o determinar que no existe.

En general, si la instancia es factible con óptimo OPT, el algoritmo se detiene durante la exploración del tamaño $k = \mathrm{OPT}$, después de descartar todas las combinaciones de tamaño menor:

$$\sum_{k=1}^{\mathrm{OPT}-1} \binom{n}{k} \text{ combinaciones descartadas } < \text{ combinaciones examinadas } \leq \sum_{k=1}^{\mathrm{OPT}} \binom{n}{k}.$$

Por ejemplo, en la instancia oficial ($n = 20$, $\mathrm{OPT} = 4$; Sección 5.4) se descartan $\binom{20}{1} + \binom{20}{2} + \binom{20}{3} = 20 + 190 + 1140 = 1350$ combinaciones antes de llegar al tamaño 4, que tiene $\binom{20}{4} = 4845$ combinaciones.

5. Algoritmo aproximado voraz (Greedy), función de eficiencia y cota H_r

5.1 Estrategia voraz implementada

El algoritmo implementado en src/auditoria/greedy.py sigue el esquema voraz clásico para Cobertura de Conjuntos:

1. Verifica inicialmente la factibilidad comprobando si $\bigcup_{i=1}^{n} S_i = \mathcal{U}$.
2. Mantiene el conjunto de categorías ya cubiertas cubiertas (K, inicialmente $\emptyset$) y la lista de interacciones elegidas seleccionadas (C, inicialmente vacía).
3. Mientras cubiertas no contenga a todo universo ($K \neq \mathcal{U}$):
    - Recorre las n interacciones candidatas y, para cada interacción no seleccionada S_i, calcula el conjunto de categorías **nuevas** que aportaría: $\Delta_i = S_i \backslash K$ (nuevos = riesgos - cubiertas).
    - Elige la interacción i^* que maximiza $|\Delta_i|$ (mejor_cantidad).
    - Agrega i^* a seleccionadas y actualiza $K \leftarrow K \cup S_{i^*}$.

5.2 Deducción formal paso a paso de la función de eficiencia $T(n, r) = O(n \cdot r^2)$

Deduzcamos rigurosamente el costo temporal de greedy(universo, interacciones) contando las operaciones en cada fase. Las operaciones sobre set (update, diferencia, issubset) se analizan bajo el supuesto habitual de costo promedio $O(1)$ por elemento para las operaciones hash.

Fase 1: Verificación de factibilidad inicial (factibilidad)

- Recorre las n interacciones de interacciones.values().
- En cada iteración ejecuta union_total.update(riesgos), procesando a lo sumo $|S_i| \leq r$ elementos.
- Al finalizar, comprueba universo.issubset(union_total) en $O(r)$.
- **Costo de la Fase 1:**

$$T_{\text{fact}}(n, r) = \sum_{i=1}^{n} O\left(|S_i|\right) + O(r) = O(n \cdot r) + O(r) = O(n \cdot r).$$

SET COVER

Fase 2: Bucle principal de selección voraz (while not universo.issubset(cubiertas))

1. **Número máximo de iteraciones del bucle while:** En cada iteración del ciclo while, dado que la instancia ya fue validada como factible, la interacción elegida i^* debe cubrir al menos una categoría de riesgo que aún no estaba en cubiertas ($|\Delta_{i^*}| \geq 1$). Como el universo $\mathcal{U}$ tiene exactamente r categorías de riesgo y en cada paso se cubre al menos 1 categoría nueva, el ciclo while puede ejecutarse **a lo sumo** $\min(n, r) \leq r$ **veces** en el peor caso (una iteración por cada categoría de riesgo que falta por cubrir).

2. **Costo de cada iteración (una elección voraz):** Dentro de una iteración fija $t \in \{1, \dots, r\}$ del bucle while:

   - Condición de parada universo.issubset(cubiertas): compara hasta r elementos, con costo $O(r)$.
   - Bucle interno for nombre, riesgos in interacciones.items(): revisa las n interacciones candidatas. Para cada una de las n interacciones:
     - Verificar si ya fue elegida (if nombre in seleccionadas): seleccionadas es una lista de Python, por lo que esta búsqueda es lineal en su longitud. Al inicio de la iteración t la lista contiene $t - 1 < \min(n, r)$ elementos, de modo que la comprobación cuesta $O(\min(n, r)) \subseteq O(r)$. *(Si se mantiene, además de la lista, un set auxiliar con las interacciones seleccionadas, la búsqueda tiene costo promedio $O(1)$; en ambos casos la cota por interacción sigue siendo $O(r)$).*
     - Calcular la diferencia de conjuntos nuevos = riesgos - cubiertas ($S_i \setminus K$) requiere examinar los elementos de S_i (a lo sumo r categorías) contra cubiertas, con costo $O(r)$.
     - Por tanto, el costo por interacción es $O(r)$.
     - Obtener cantidad = len(nuevos) y comparar if cantidad > mejor_cantidad cuesta $O(1)$.
   - Por lo tanto, revisar las n interacciones restantes para decidir cuál cubre más categorías nuevas tiene un costo por elección de:

$$T_{\text{elección}}(n, r) = \sum_{i=1}^{n} O\,(r) = O(n \cdot r).$$

- Actualizar seleccionadas.append(mejor_interaccion) y cubiertas.update(nuevos) al final de la iteración toma $O(r)$.

3. **Suma sobre todas las iteraciones en el peor caso:** Multiplicando el número máximo de iteraciones (r elecciones en el peor caso) por el costo de cada elección ($O(n \cdot r)$):

$$T_{\text{bucle}}(n,r) = \sum_{t=1}^{r} O\,(n \cdot r) = r \cdot O(n \cdot r) = O(n \cdot r^2).$$

Costo total del algoritmo voraz

Sumando la Fase 1 y la Fase 2:

$$T_{\text{greedy}}(n,r) = T_{\text{fact}}(n,r) + T_{\text{bucle}}(n,r) = O(n \cdot r) + O(n \cdot r^2) = O(n \cdot r^2).$$

En consecuencia, queda demostrado formalmente que la función de eficiencia del algoritmo voraz en el peor caso es $T(n,r) = O(n \cdot r^2)$ (y más específicamente $O(n \cdot r \cdot \min(n,r))$), lo cual es polinomial en el tamaño de la entrada.

5.3 Garantía teórica de aproximación: Cota armónica H_r

Aunque el algoritmo voraz no garantiza encontrar siempre el óptimo exacto, posee una garantía matemática demostrada por **Johnson (1974)** y **Chvátal (1979)**.

Teorema 1 (Razón de aproximación del algoritmo voraz para Set Cover — Johnson, 1974; Chvátal, 1979). *Sea $I = (\mathcal{U}, \mathcal{I})$ una instancia factible de Cobertura Mínima de Auditoría con $|\mathcal{U}| = r$ categorías de riesgo. Sea C^* una solución óptima de cardinalidad mínima* $\text{OPT} = |C^*|$, *y sea C_{greedy} la solución devuelta por el algoritmo voraz con cardinalidad* $\text{SOL} = |C_{\text{greedy}}|$. *Entonces:*

$$\text{SOL} \leq H_r \cdot \text{OPT},$$

donde H_r es el r-ésimo número armónico:

$$H_r = \sum_{j=1}^{r} \frac{1}{j} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{r} \leq \ln(r) + 1.$$

SET COVER

Demostración mediante el argumento de distribución de precios (Chvátal, 1979)

Cada vez que el algoritmo voraz selecciona una interacción en la iteración t, esta tiene un "costo" unitario de 1 (pues cuenta como 1 interacción adicional auditada). Distribuimos ese costo 1 equitativamente entre las $|\Delta_t|$ categorías de riesgo **nuevas** que quedan cubiertas por primera vez en esa iteración. Así, a cada categoría $R \in \Delta_t$ cubierta en el paso t se le asigna un precio:

$$p(R) = \frac{1}{|\Delta_t|}.$$

Como cada categoría de $\mathcal{U}$ se cubre exactamente por primera vez en algún paso, la suma de los precios de todas las categorías es igual al número total de interacciones seleccionadas por el algoritmo voraz:

$$\text{SOL} = |C_{\text{greedy}}| = \sum_{j=1}^{r} p\,(R_{\pi(j)}),$$

donde $R_{\pi(1)}, R_{\pi(2)}, \dots, R_{\pi(r)}$ es el orden en el que las categorías de riesgo fueron cubiertas por el algoritmo voraz.

Precisión sobre el orden. Una misma interacción puede cubrir varias categorías nuevas en una sola iteración t. Todas las categorías de ese grupo Δ_t reciben el mismo precio $1/|\Delta_t|$ y, para efectos del análisis, se ordenan de forma arbitraria dentro del grupo. Los grupos se ordenan según la iteración en que fueron cubiertos. Esto define el orden π sin suponer que cada categoría se cubre en una iteración diferente.

Sea $R_{\pi(j)}$ la j-ésima categoría en ese orden y sea t la iteración en la que fue cubierta. Al **inicio** de la iteración t, todas las categorías $R_{\pi(j)}, R_{\pi(j+1)}, \dots, R_{\pi(r)}$ siguen sin cubrir (las del mismo grupo que preceden a $R_{\pi(j)}$ en el orden arbitrario tampoco estaban cubiertas, lo que solo aumenta el conteo). Por tanto, el número de categorías sin cubrir al inicio de la iteración t es:

$$u_j \geq r - (j-1) = r - j + 1$$

categorías de riesgo aún sin cubrir en $\mathcal{U} \backslash K$ al inicio de la iteración t.

Como la solución óptima C^* cubre todo $\mathcal{U}$ usando OPT interacciones, también cubre esas u_j categorías restantes con a lo sumo OPT conjuntos. Por el principio del palomar (promedio), al menos una de las interacciones de C^* cubre como mínimo $\frac{u_j}{\text{OPT}}$ categorías aún no cubiertas. Esa interacción no puede haber sido seleccionada antes (aporta categorías nuevas), por lo que es una candidata válida en la iteración t.

Dado que el algoritmo voraz elige en cada paso la interacción que maximiza el número de categorías nuevas cubiertas ($|\Delta_t|$), la interacción elegida por el voraz cubre al menos tantas categorías nuevas como la mejor interacción de C^*:

$$|\Delta_t| \geq \frac{u_j}{\text{OPT}} \geq \frac{r-j+1}{\text{OPT}}.$$

Por lo tanto, el precio asignado a la j-ésima categoría cubierta cumple:

$$p(R_{\pi(j)}) = \frac{1}{|\Delta_t|} \leq \frac{\text{OPT}}{r-j+1}.$$

Sumando sobre todas las categorías $j = 1,2,\dots,r$:

$$\text{SOL} = \sum_{j=1}^{r} p\,(R_{\pi(j)}) \leq \sum_{j=1}^{r} \frac{\text{OPT}}{r-j+1} = \text{OPT} \sum_{m=1}^{r} \frac{1}{m} = H_r \cdot \text{OPT}.$$

Por comparación integral con la función monótona decreciente $f(x) = 1/x$:

$$H_r = 1 + \sum_{m=2}^{r} \frac{1}{m} \leq 1 + \int_1^r \frac{1}{x}\,dx = \ln(r) + 1.$$

Concluimos así que la razón de aproximación del algoritmo voraz satisface:

$$\rho(r) = \frac{\text{SOL}}{\text{OPT}} \leq H_r \leq \ln(r) + 1.$$

5.4 Validación experimental en la instancia oficial del Grupo 3

Se ejecutaron ambos algoritmos del repositorio (brute_force.py y greedy.py, opciones 2 y 3 del menú de src/main.py) sobre la entrada oficial data/entrada_G3_mochila_20_nodos.json, usando como conjunto S_i de cada etapa el campo riesgos_cubiertos_auditoria_completa. Los resultados se encuentran en la carpeta resultados_auditoria/.

SET COVER

Parámetros de la instancia

Dato	Valor
Número de interacciones n	20 $(E_{01}, \dots, E_{20})$
Número de categorías de riesgo r	12 $(R_{01}, \dots, R_{12})$
Máximo de categorías por interacción $d = \max_i \lvert S_i \rvert$	4
Relaciones de cobertura $L = \sum_i \lvert S_i \rvert$	72
Factibilidad $\bigcup_i S_i = \mathcal{U}$	Sí

Solución exacta (fuerza bruta)

Archivo: resultados_auditoria/datos_profe.json_fuerza_bruta_interacciones.json.

$$C^*_{\mathrm{FB}} = \{E_{01}, E_{03}, E_{05}, E_{10}\}, \qquad \mathrm{OPT} = 4.$$

Interacción	Categorías cubiertas
E_{01}	$R_{01}, R_{02}, R_{03}, R_{10}$
E_{03}	$R_{01}, R_{06}, R_{07}, R_{12}$
E_{05}	R_{04}, R_{10}, R_{11}
E_{10}	$R_{02}, R_{05}, R_{08}, R_{09}$

Unión:

$$E_{01} \cup E_{03} \cup E_{05} \cup E_{10} = \{R_{01}, R_{02}, R_{03}, R_{04}, R_{05}, R_{06}, R_{07}, R_{08}, R_{09}, R_{10}, R_{11}, R_{12}\} = \mathcal{U}.$$

Justificación de que $\mathrm{OPT} = 4$**.** No se deduce a partir de que cada interacción cubra a lo sumo 4 categorías, sino del recorrido exhaustivo: la fuerza bruta examina los tamaños en orden creciente y descartó **todas** las $\binom{20}{1} + \binom{20}{2} + \binom{20}{3} = 1350$ combinaciones de tamaño 1, 2 y 3 (ninguna cubre $\mathcal{U}$) antes de encontrar la primera cobertura de tamaño 4. Por tanto, no existe cobertura con 3 o menos interacciones y $\mathrm{OPT} = 4$.

Solución voraz

Archivo: resultados_auditoria/datos_profe.json_greedy_interacciones.json.

Iteración t	Interacción elegida	Categorías nuevas Δ_t	$\lvert\Delta_t\rvert$

SET COVER

Iteración t	Interacción elegida	Categorías nuevas Δ_t	$\lvert\Delta_t\rvert$
1	E_{01}	$R_{01}, R_{02}, R_{03}, R_{10}$	4
2	E_{16}	$R_{04}, R_{05}, R_{09}, R_{12}$	4
3	E_{06}	R_{07}, R_{08}, R_{11}	3
4	E_{03}	R_{06}	1

$$C_{\text{greedy}} = \{E_{01}, E_{16}, E_{06}, E_{03}\}, \qquad \text{SOL} = 4.$$

La suma $4 + 4 + 3 + 1 = 12 = r$ confirma que la unión es exactamente $\mathcal{U}$. En caso de empate, el algoritmo conserva la primera interacción encontrada con el máximo (comparación estricta cantidad > mejor_cantidad), lo que hace la ejecución determinista.

Las dos soluciones son conjuntos distintos, pero ambas tienen tamaño 4: el voraz encontró una solución óptima en esta instancia.

Razón de aproximación

$$\rho = \frac{\text{SOL}_{\text{greedy}}}{\text{OPT}_{\text{FB}}} = \frac{4}{4} = 1,$$

y la cota teórica para $r = 12$ es

$$H_{12} = 1 + \frac{1}{2} + \cdots + \frac{1}{12} = \frac{86021}{27720} \approx 3.1032 \qquad (\ln 12 + 1 \approx 3.4849).$$

Se verifica que

$$1 \leq \rho = 1 \leq H_{12} \approx 3.1032.$$

> **Importante.** Comprobar esta desigualdad en una instancia es una **validación experimental**, no una demostración del teorema. La garantía matemática SOL $\leq H_r \cdot$ OPT está demostrada en la Sección 5.3 para toda instancia factible; el experimento solo muestra que la instancia oficial es consistente con ella (y que en este caso el voraz alcanza el óptimo).

Tiempos registrados

El archivo resultados_auditoria/comparativa_rendimiento.csv registra, para la instancia oficial (etiqueta datos_profe.json):

Algoritmo	Tiempo (s)	Valor obtenido
Fuerza bruta	0.1127	4
Voraz	0.0020	4

6. Referencias bibliográficas

1. **Karp, R. M.** (1972). *Reducibility among combinatorial problems*. En R. E. Miller & J. W. Thatcher (Eds.), Complexity of Computer Computations (pp. 85–103). Plenum Press.
2. **Johnson, D. S.** (1974). *Approximation algorithms for combinatorial problems*. Journal of Computer and System Sciences, 9(3), 256–278.
3. **Chvátal, V.** (1979). *A greedy heuristic for the set-covering problem*. Mathematics of Operations Research, 4(3), 233–235.
4. **Garey, M. R., & Johnson, D. S.** (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
5. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C.** (2022). *Introduction to Algorithms* (4th ed., Cap. 34–35: NP-Completeness & Approximation Algorithms). MIT Press.
6. **Garey, M. R., Johnson, D. S., & Stockmeyer, L.** (1976). *Some simplified NP-complete graph problems*. Theoretical Computer Science, 1(3), 237–267.